IT5008: Tutorial 5 — Relational Algebra

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1



The Book Exchange Schema (Recap)

Same NUN Book Exchange scenario and schema as Tutorials 3 and 4.

- **department**(department, faculty)
- **student**(name, email, year, department *(FK)*, graduate)
- **book**(title, format, pages, language, authors, publisher, year, edition, ISBN10 *(unique)*, ISBN13)
- **copy**(owner, book, copy) — FKs to `student` and `book`
- **loan**(borrower, owner, book, copy, borrowed, returned)

New this tutorial

Writing queries as **relational algebra** expressions, and translating them to/from SQL — including **universal quantification** without `NOT EXISTS`.

Relational Algebra Cheat Sheet

Operator	**Notation**	**SQL counterpart**
Rename	$\rho(R, r)$	`FROM R r`
Selection	$\sigma_{[c]}(R)$	`WHERE c`
Projection	$\pi_{[a_1,\ldots]}(R)$	`SELECT DISTINCT a1, ...`
Cross product	$R \times S$	`FROM R, S`
Join	$R \bowtie_{[c]} S$	`FROM R, S WHERE c`
Union / Difference	$R \cup S$, $R - S$	`UNION`, `EXCEPT`

- Relational algebra works on **sets**: π removes duplicates automatically — hence `DISTINCT` (or a set operator) in the SQL.
- $R \bowtie_{[c]} S \equiv \sigma_{[c]}(R \times S)$.
- $\cup$ and $-$ need **union-compatible** inputs (same number and types of columns).

Preliminary: Basic Relational Algebra

Not discussed in class — attempt before the tutorial, since later questions build on these.

1. **Basic Relational Algebra.**
   (a) Find the different departments in the School of Computing.
   (b) Print the emails of the students who *borrowed* but did *not lend* a copy of a book on the day that they joined the university.

1(a). Departments in the School of Computing

Rename, select the faculty, project the department.

$$\pi_{[\text{d.department}]}\Big(\sigma_{[\text{d.faculty} = \text{'School of Computing'}]}\big(\rho(\text{department}, \text{d})\big)\Big)$$

```
1 SELECT d.department
2 FROM department d
3 WHERE d.faculty = 'School of Computing';
```

`department` is the primary key, so no duplicates can arise — no `DISTINCT` needed.

1(b). Borrowed but Did Not Lend on Joining Day

"Borrowed on day one" **minus** "lent on day one" — a set difference.

$$\pi_{[\text{s1.email}]}\Big(\sigma_{[\text{s1.email} = \text{l1.borrower} \wedge \text{l1.borrowed} = \text{s1.year}]}\big(\rho(\text{student}, \text{s1}) \times \rho(\text{loan}, \text{l1})\big)\Big)$$
$$- \ \pi_{[\text{s2.email}]}\Big(\sigma_{[\text{s2.email} = \text{l2.owner} \wedge \text{l2.borrowed} = \text{s2.year}]}\big(\rho(\text{student}, \text{s2}) \times \rho(\text{loan}, \text{l2})\big)\Big)$$

```
1  SELECT s1.email
2  FROM loan l1, student s1
3  WHERE s1.email = l1.borrower AND l1.borrowed = s1.year
4  EXCEPT
5  SELECT s2.email
6  FROM loan l2, student s2
7  WHERE s2.email = l2.owner AND l2.borrowed = s2.year;
```

`EXCEPT` is a set operator: it already removes duplicates.

Discussion: Integrity Check & Universal Quantification

To be discussed in class — attempt before the tutorial; you may be called on to present.

2. **Relational Algebra.** Let us check the integrity of the data. Find the different emails of the students who borrowed *or* lent a copy of a book *before* they joined the university. There should not be any.
   (a) Solve the problem **with** a set operation.
   (b) Solve the problem **without** a set operation.
3. **Universal Quantification.** Print the emails and the names of the different students who borrowed *all* the books authored by Adam Smith.

2(a). Integrity Check — With a Set Operation

"Borrowed too early" **or** "lent too early" — a union.

$$\pi_{[\text{s1.email}]}\Big(\sigma_{[\text{s1.email} = \text{l1.borrower} \wedge \text{l1.borrowed} < \text{s1.year}]}\big(\rho(\text{student}, \text{s1}) \times \rho(\text{loan}, \text{l1})\big)\Big)$$
$$\cup\ \pi_{[\text{s2.email}]}\Big(\sigma_{[\text{s2.email} = \text{l2.owner} \wedge \text{l2.borrowed} < \text{s2.year}]}\big(\rho(\text{student}, \text{s2}) \times \rho(\text{loan}, \text{l2})\big)\Big)$$

```
1  SELECT s1.email
2  FROM loan l1, student s1
3  WHERE s1.email = l1.borrower AND l1.borrowed < s1.year
4  UNION
5  SELECT s2.email
6  FROM loan l2, student s2
7  WHERE s2.email = l2.owner AND l2.borrowed < s2.year;
```

Compare with 1(b): same shape, but $-$ becomes $\cup$ and $=$ becomes $<$.

2(b). Integrity Check — Without a Set Operation

Fold the "or" into a **single selection** with a disjunction.

$$\pi_{[\text{s.email}]}\Big(\sigma_{[(\text{s.email} = \text{l.borrower} \vee \text{s.email} = \text{l.owner}) \wedge \text{l.borrowed} < \text{s.year}]} \big(\rho(\text{student}, \text{s}) \times \rho(\text{loan}, \text{l})\big)\Big)$$

```
1  SELECT DISTINCT s.email
2  FROM student s, loan l
3  WHERE (s.email = l.borrower OR s.email = l.owner)
4    AND l.borrowed < s.year;
```

- `DISTINCT` **is now required**: without a set operator, SQL keeps duplicates, but π does not.
- **Mind the parentheses**: `AND` binds tighter than `OR`.

3. Borrowed *All* of Adam Smith's Books

In Tutorial 4 we wrote this with a **double `NOT EXISTS`**. But `NOT EXISTS` has no direct relational algebra counterpart — we need to rephrase it with **set difference**.

Break it down:

1. Find the students for whom there is **at least one** Adam Smith book they have *not* borrowed.
2. Answer = **all** students (including those who borrowed nothing) **minus** the students from Step 1.

Step 1, split further:

A. Every *possible* (student, Adam Smith book) pair — a cross product.

B. Every *actual* (student, Adam Smith book) borrowing — a join with `loan`.

C. A − B are the **missing** pairs; project to keep only email and name.

3. The Relational Algebra

We use $:=$ to name sub-expressions we refer to later.

$$Q_{1A} := \pi_{[\text{s1.email, s1.name}]}\Big(\rho(\text{student}, \text{s1})\Big) \times \pi_{[\text{b1.ISBN13}]}\Big(\sigma_{[\text{b1.authors} = \text{'Adam Smith'}]}\Big(\rho(\text{book}, \text{b1})\Big)\Big)$$

$$Q_{1B} := \pi_{[\text{s2.email, s2.name, b2.ISBN13}]}\Big(\rho(\text{loan}, \text{l2}) \bowtie_{[\text{l2.book} = \text{b2.ISBN13} \wedge \text{b2.authors} = \text{'Adam Smith'}]} \rho(\text{book}, \text{b2}) \bowtie_{[\text{l2.borrower} = \text{s2.email}]} \rho(\text{student}, \text{s2})\Big)$$

$$Q_1 := \pi_{[\text{s2.email, s2.name}]}\Big(Q_{1A} - Q_{1B}\Big)$$

$$Q_2 := \pi_{[\text{s3.email, s3.name}]}\Big(\rho(\text{student}, \text{s3})\Big) - Q_1$$

- Q_{1A} and Q_{1B} are **union-compatible**: both are (email, name, ISBN13).
- $Q_{1A} - Q_{1B}$: the (student, book) pairs that *should* exist but *don't*.

3. Translating Back to SQL

Each $-$ becomes an `EXCEPT`; Q_1 becomes a subquery in `FROM`.

```
1  SELECT s3.email, s3.name
2  FROM student s3
3  EXCEPT
4  SELECT tmp.email, tmp.name
5  FROM (
6    SELECT s1.email, s1.name, b1.ISBN13        -- Q1A
7    FROM student s1, book b1
8    WHERE b1.authors = 'Adam Smith'
9    EXCEPT
10   SELECT s2.email, s2.name, b2.ISBN13        -- Q1B
11   FROM student s2, book b2, loan l2
12   WHERE b2.authors = 'Adam Smith'
13     AND s2.email = l2.borrower
14     AND b2.ISBN13 = l2.book
15 ) AS tmp;
```

Same answer as Tutorial 4's double-`NOT EXISTS` query: "no Adam Smith book is missing from s's loans."

3. Why This Is the Same as `NOT EXISTS`

Nested SQL (Tut 4)	**Relational algebra (Tut 5)**
inner `NOT EXISTS` (no loan of b by s)	$Q_{1A} - Q_{1B}$
outer `NOT EXISTS` (no such b for s)	$\pi(\text{student}) - Q_1$

- $\forall b\,.\,\text{borrowed}(s, b) \equiv \neg\exists b\,.\,\neg\text{borrowed}(s, b)$
- Each **negation** becomes a **set difference** against the set of all candidates.
- Students who borrowed *nothing* still appear in Q_{1A} (via the cross product), so they correctly land in Q_1 and are excluded — unless Adam Smith has no books, in which case **every** student qualifies (vacuous truth).

Guidelines & Common Pitfalls

- **Sets vs. bags.** π eliminates duplicates; a plain SQL `SELECT` does not. Add `DISTINCT` unless a set operator (or a key) already guarantees uniqueness (1(a), 2(b)).
- **"or" $\rightarrow \cup$ or $\vee$; "but not" $\rightarrow -$.** Either a set operator between two queries, or a combined condition inside one σ (Q2).
- **Parenthesise disjunctions.** `(A OR B) AND C` $\neq$ `A OR B AND C` (2(b)).
- **"For all" has no single operator.** Build all candidates ($\times$), remove what actually happened ($-$), then remove the failures from everyone ($-$ again) (Q3).
- **Union-compatibility.** Both sides of $\cup$ / $-$ must have the same columns — keep `ISBN13` in Q_{1A} and Q_{1B} *until after* the difference.

Questions?
Drop a mail at: pratik.karmakar@u.nus.edu